1

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: priority_enc Design
//
////////////////////////////////////////////////////////////////////////////////
module enc_top();

    // 1. Generate the clock
    bit clk;
    initial begin
    clk = 0;
    forever #5 clk = ~clk;
    end

    // 2. instantiate the interface, and pass the clock
    enc_if e_if(clk);

    // 3. instantiate the tb, DUT, monitor, and pass the interface
    priority_enc_tb tb(e_if);

    priority_enc dut(e_if);

    enc_monitor monitor(e_if);

    // 4. bind the SVA module to the design, and pass the interface
    bind priority_enc enc_sva enc_if_inst(e_if);

endmodule
```

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: priority_enc Design
//
////////////////////////////////////////////////////////////////////////////////
interface enc_if(clk);

    // 2. Add the clock as an input port
    input clk;

    // 3. Add the internal signals of the interface
    logic clk;
    logic rst;
    logic [3:0] D;
    logic  [1:0] Y;
    logic valid;

    // 4. Add the modports
    modport DUT (input clk, rst, D, output Y, valid);

    modport TEST (output rst, D, input clk, Y, valid);

    modport MONITOR(input clk, rst, D, Y, valid);

endinterface
```

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: priority_enc Design
//
////////////////////////////////////////////////////////////////////////////////
module priority_enc (enc_if.DUT e_if);

always @(posedge e_if.clk) begin
  if (e_if.rst)
     e_if.Y <= 2'b0;
  else
    casex (e_if.D)
        4'b1000: e_if.Y <= 0;
        4'bX100: e_if.Y <= 1;
        4'bXX10: e_if.Y <= 2;
        4'bXXX1: e_if.Y <= 3;
        default: e_if.Y <= 0;
    endcase
    e_if.valid <= (~|e_if.D)? 1'b0: 1'b1;
end
endmodule
```

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: priority_enc Design
//
////////////////////////////////////////////////////////////////////////////////
module enc_monitor(enc_if.MONITOR e_if);

    initial begin
        $monitor("rst = %b, D = %b, Y = %b valid = %b",e_if.rst, e_if.D, e_if.Y, e_if.valid);
    end

endmodule
```

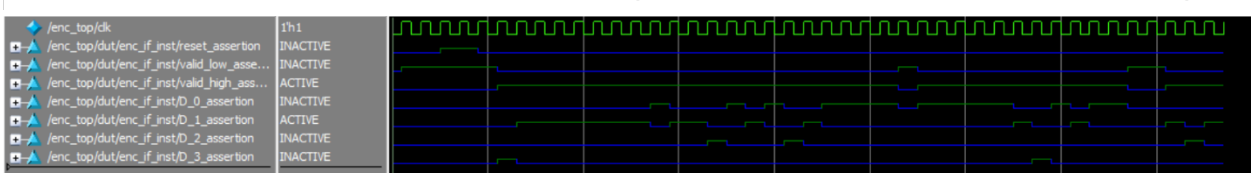
```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: priority_enc Design
//
////////////////////////////////////////////////////////////////////////////////

module enc_sva(enc_if.DUT e_if);

    //
    property reset_active;
        @(posedge e_if.clk) e_if.rst |=> (e_if.Y == 2'b00);
    endproperty

    reset_assertion : assert property (reset_active);
    reset_coverage : cover property (reset_active);

    //
    property valid_low_check;
        @(posedge e_if.clk)  (~|e_if.D == 1) |=> (e_if.valid == 0);
    endproperty

    valid_low_assertion: assert property (valid_low_check);
    valid_low_coverage: cover property (valid_low_check);

    property valid_high_check;
        @(posedge e_if.clk)  (|e_if.D == 1) |=> (e_if.valid == 1);
    endproperty

    valid_high_assertion: assert property (valid_high_check);
    valid_high_coverage: cover property (valid_high_check);
```

```
    property D_0;
        @(posedge e_if.clk) disable iff (e_if.rst) e_if.D[0] |=> (e_if.Y == 3);
    endproperty

    D_0_assertion: assert property (D_0);
    D_0_cover: cover property (D_0);

    property D_1;
        @(posedge e_if.clk) disable iff (e_if.rst) (!e_if.D[0] && e_if.D[1]) |=> (e_if.Y == 2);
    endproperty

    D_1_assertion: assert property (D_1);
    D_1_cover: cover property (D_1);

    property D_2;
        @(posedge e_if.clk) disable iff (e_if.rst) (!e_if.D[0] && !e_if.D[1] && e_if.D[2]) |=> (e_if.Y == 1);
    endproperty

    D_2_assertion: assert property (D_2);
    D_2_cover: cover property (D_2);

    property D_3;
        @(posedge e_if.clk) disable iff (e_if.rst) (!e_if.D[0] && !e_if.D[1] && !e_if.D[2] && e_if.D[3]) |=> (e_if.Y == 0);
    endproperty

    D_3_assertion: assert property (D_3);
    D_3_cover: cover property (D_3);

endmodule
```

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: priority_enc Design
//
////////////////////////////////////////////////////////////////////////////////

module priority_enc_tb (enc_if.TEST e_if);

    // reset initial values for inputs
    initial begin

        e_if.rst = 0;
        e_if.D = 0;

        // Reset
        repeat(2) @(posedge e_if.clk);
        e_if.rst = 1;
        repeat(2) @(posedge e_if.clk);
        e_if.rst = 0;

        for(int i = 0; i < 40; i++) begin
            @(posedge e_if.clk);
            e_if.D = $urandom_range(0,15);
        end

    $stop;
    end

endmodule
```

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
/enc_top/dut/enc_if_inst/reset_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0
/enc_top/dut/enc_if_inst/valid_low_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0
/enc_top/dut/enc_if_inst/valid_high_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0
/enc_top/dut/enc_if_inst/D_0_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0
/enc_top/dut/enc_if_inst/D_1_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0
/enc_top/dut/enc_if_inst/D_2_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0
/enc_top/dut/enc_if_inst/D_3_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included
/enc_top/dut/enc_if_inst/reset_coverage	SVA	✓	Off	2	1	Unli...	1	100%		✓
/enc_top/dut/enc_if_inst/valid_low_coverage	SVA	✓	Off	8	1	Unli...	1	100%		✓
/enc_top/dut/enc_if_inst/valid_high_coverage	SVA	✓	Off	34	1	Unli...	1	100%		✓
/enc_top/dut/enc_if_inst/D_0_cover	SVA	✓	Off	15	1	Unli...	1	100%		✓
/enc_top/dut/enc_if_inst/D_1_cover	SVA	✓	Off	14	1	Unli...	1	100%		✓
/enc_top/dut/enc_if_inst/D_2_cover	SVA	✓	Off	3	1	Unli...	1	100%		✓
/enc_top/dut/enc_if_inst/D_3_cover	SVA	✓	Off	2	1	Unli...	1	100%		✓



```
================================================================================
=== Instance: /enc_top/e_if
=== Design Unit: work.enc_if
================================================================================
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         18        18         0   100.00%

================================Toggle Details================================
```

```
================================================================================
=== Instance: /enc_top/dut
=== Design Unit: work.priority_enc
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         7         7         0   100.00%

================================Branch Details================================
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       8         8         0   100.00%

================================Statement Details================================

Statement Coverage for instance /enc_top/dut --
```

```
DIRECTIVE COVERAGE:
--------------------------------------------------------------------------------------
Name                                      Design Design   Lang File(Line)      Hits Status
                                          Unit   UnitType
--------------------------------------------------------------------------------------
/enc_top/dut/enc_if_inst/reset_coverage  enc_sva Verilog  SVA  enc_sva.sv(17)     2 Covered
/enc_top/dut/enc_if_inst/valid_low_coverage
                                          enc_sva Verilog  SVA  enc_sva.sv(25)     8 Covered
/enc_top/dut/enc_if_inst/valid_high_coverage
                                          enc_sva Verilog  SVA  enc_sva.sv(32)    34 Covered
/enc_top/dut/enc_if_inst/D_0_cover        enc_sva Verilog  SVA  enc_sva.sv(39)    15 Covered
/enc_top/dut/enc_if_inst/D_1_cover        enc_sva Verilog  SVA  enc_sva.sv(46)    14 Covered
/enc_top/dut/enc_if_inst/D_2_cover        enc_sva Verilog  SVA  enc_sva.sv(53)     3 Covered
/enc_top/dut/enc_if_inst/D_3_cover        enc_sva Verilog  SVA  enc_sva.sv(60)     2 Covered

TOTAL DIRECTIVE COVERAGE: 100.00%  COVERS: 7
```

```
ASSERTION RESULTS:
--------------------------------------------------------------------
Name                    File(Line)                    Failure      Pass
                                                      Count        Count
--------------------------------------------------------------------
/enc_top/dut/enc_if_inst/reset_assertion
                        enc_sva.sv(16)                  0            1
/enc_top/dut/enc_if_inst/valid_low_assertion
                        enc_sva.sv(24)                  0            1
/enc_top/dut/enc_if_inst/valid_high_assertion
                        enc_sva.sv(31)                  0            1
/enc_top/dut/enc_if_inst/D_0_assertion
                        enc_sva.sv(38)                  0            1
/enc_top/dut/enc_if_inst/D_1_assertion
                        enc_sva.sv(45)                  0            1
/enc_top/dut/enc_if_inst/D_2_assertion
                        enc_sva.sv(52)                  0            1
/enc_top/dut/enc_if_inst/D_3_assertion
                        enc_sva.sv(59)                  0            1

Total Coverage By Instance (filtered view): 99.23%
```

2

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: ALU Design
//
////////////////////////////////////////////////////////////////////////////////
module ALU_top ();

    // 1. Generate the clock
    bit clk;
    initial begin
    clk = 0;
    forever #5 clk = ~clk;
    end

    // 2. instantiate the interface, and pass the clock
    ALU_if alu_inst(clk);

    // 3. instantiate the tb, DUT, monitor, and pass the interface
    ALU_tb tb(alu_inst);

    ALU_4_bit dut(alu_inst);

    ALU_monitor monitor(alu_inst);

    // 4. bind the SVA module to the design, and pass the interface
    bind ALU_4_bit ALU_sva ALU_if_inst(alu_inst);

endmodule
```

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: ALU Design
//
////////////////////////////////////////////////////////////////////////////////
interface ALU_if(clk);

    parameter  Add            = 2'b00; // A + B
    parameter  Sub            = 2'b01; // A - B
    parameter  Not_A          = 2'b10; // ~A
    parameter  ReductionOR_B  = 2'b11; // |B

    // 2. Add the clock as an input port
    input clk;

    // 3. Add the internal signals of the interface
    logic reset;
    logic [1:0] Opcode; // The opcode
    logic signed [3:0] A;   // Input data A in 2's complement
    logic signed [3:0] B;   // Input data B in 2's complement
    logic signed [4:0] C; // ALU output in 2's complement

    // 4. Add the modports
    modport DUT(input clk, reset, Opcode, A, B, output C);

    modport TEST(output reset, Opcode, A, B, input clk, C);

    modport MONITOR(input clk, reset, Opcode, A, B, C);

endinterface
```

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: ALU Design
//
////////////////////////////////////////////////////////////////////////////////

module ALU_4_bit(ALU_if.DUT alu_inst);

    logic signed [4:0] Alu_out; // ALU output in 2's complement

    // Do the operation
    always @* begin
        case (alu_inst.Opcode)
            alu_inst.Add:            Alu_out = alu_inst.A + alu_inst.B;
            alu_inst.Sub:            Alu_out = alu_inst.A - alu_inst.B;
            alu_inst.Not_A:          Alu_out = ~alu_inst.A;
            alu_inst.ReductionOR_B:  Alu_out = |alu_inst.B;
          default:  Alu_out = 5'b0;
        endcase
    end // always @ *

    // Register output C
    always @(posedge alu_inst.clk or posedge alu_inst.reset) begin
        if (alu_inst.reset)
          alu_inst.C <= 5'b0;
        else
          alu_inst.C<= Alu_out;
    end

endmodule
```

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: ALU Design
//
////////////////////////////////////////////////////////////////////////////////
module ALU_sva(ALU_if.DUT alu_inst);

    // always_comb begin : reset_check
    //     if(alu_inst.reset)
    //         assert(alu_inst.C == 5'b0);
    // end

    property reset_check;
        @(posedge alu_inst.clk) (alu_inst.reset) |=> (alu_inst.C == 5'b0);
    endproperty

    reset_assertion: assert property(reset_check);
    reset_coverage: cover property(reset_check);

    property alu_out_add;
        @(posedge alu_inst.clk) disable iff(alu_inst.reset) (alu_inst.Opcode == alu_inst.Add) |=> (alu_inst.C == $past(alu_inst.A) + $past(alu_inst.B));
    endproperty

    alu_add_assertion: assert property(alu_out_add);
    alu_add_coverage: cover property(alu_out_add);

    property alu_out_Sub;
        @(posedge alu_inst.clk) disable iff(alu_inst.reset) (alu_inst.Opcode == alu_inst.Sub) |=> (alu_inst.C == $past(alu_inst.A) - $past(alu_inst.B));
    endproperty

    alu_Sub_assertion: assert property(alu_out_Sub);
    alu_Sub_coverage: cover property(alu_out_Sub);
```

```
    property alu_out_Not_A;
        @(posedge alu_inst.clk) disable iff(alu_inst.reset) (alu_inst.Opcode == alu_inst.Not_A) |=> (alu_inst.C == ~$past(alu_inst.A));
    endproperty

    alu_Not_A_assertion: assert property(alu_out_Not_A);
    alu_Not_A_coverage: cover property(alu_out_Not_A);

    property alu_out_ReductionOR_B;
        @(posedge alu_inst.clk) disable iff(alu_inst.reset) (alu_inst.Opcode == alu_inst.ReductionOR_B) |=> (alu_inst.C == |$past(alu_inst.B));
    endproperty

    alu_ReductionOR_B_assertion: assert property(alu_out_ReductionOR_B);
    alu_ReductionOR_B_coverage: cover property(alu_out_ReductionOR_B);

endmodule
```

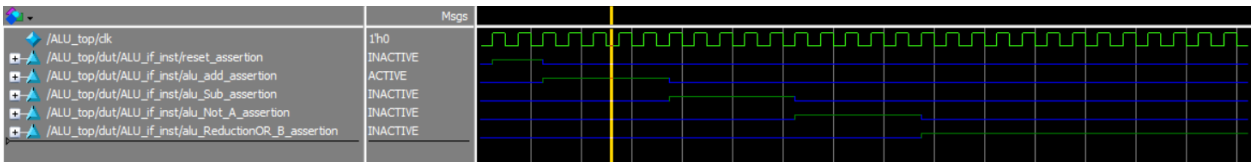
```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: ALU Design
//
////////////////////////////////////////////////////////////////////////////////
module ALU_monitor(ALU_if.MONITOR alu_inst);

    initial begin
        $monitor("reset = %b opcode = %b A = %b B = %b C = %b",alu_inst.reset, alu_inst.Opcode, alu_inst.A, alu_inst.B, alu_inst.C);
    end

endmodule
```

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: ALU Design
//
////////////////////////////////////////////////////////////////////////////////


module ALU_tb(ALU_if.TEST alu_inst);

    // reset initial values for inputs
    initial begin

        alu_inst.reset = 0;
        #2;
        alu_inst.reset = 1;
        repeat(2) @(posedge alu_inst.clk);
        alu_inst.reset = 0;

        alu_inst.Opcode = 0;
        alu_inst.A = 0;
        alu_inst.B = 0;

        // Add operation
        alu_inst.Opcode = alu_inst.Add;
        for(int i = 0; i < 4; i++) begin
            @(posedge alu_inst.clk);
            alu_inst.A = $random;
            alu_inst.B = $random;
        end

        repeat(2) @(negedge alu_inst.clk);
```

```
        // Sub operation
        alu_inst.Opcode = alu_inst.Sub;
        for(int i = 0; i < 4; i++) begin
            @(posedge alu_inst.clk);
            alu_inst.A = $random;
            alu_inst.B = $random;
        end

        repeat(2) @(negedge alu_inst.clk);

        // Not_A operation
        alu_inst.Opcode = alu_inst.Not_A;
        for(int i = 0; i < 4; i++) begin
            @(posedge alu_inst.clk);
            alu_inst.A = $random;
            alu_inst.B = $random;
        end

        repeat(2) @(negedge alu_inst.clk);

        // ReductionOR_B operation
        alu_inst.Opcode = alu_inst.ReductionOR_B;
        for(int i = 0; i < 4; i++) begin
            @(posedge alu_inst.clk);
            alu_inst.A = $random;
            alu_inst.B = $random;
        end

        #100; $stop;
    end
endmodule
```

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time
/ALU_top/dut/ALU_if_inst/reset_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns
/ALU_top/dut/ALU_if_inst/alu_add_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns
/ALU_top/dut/ALU_if_inst/alu_Sub_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns
/ALU_top/dut/ALU_if_inst/alu_Not_A_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns
/ALU_top/dut/ALU_if_inst/alu_ReductionOR_B_assertion	Concurrent	SVA	on	0	1	-	0B	0B	0 ns

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory
/ALU_top/dut/ALU_if_inst/reset_coverage	SVA	✓	Off	2	1	Unlimited	1	100%		✓	0
/ALU_top/dut/ALU_if_inst/alu_add_coverage	SVA	✓	Off	5	1	Unlimited	1	100%		✓	0
/ALU_top/dut/ALU_if_inst/alu_Sub_coverage	SVA	✓	Off	5	1	Unlimited	1	100%		✓	0
/ALU_top/dut/ALU_if_inst/alu_Not_A_coverage	SVA	✓	Off	5	1	Unlimited	1	100%		✓	0
/ALU_top/dut/ALU_if_inst/alu_ReductionOR_B_coverage	SVA	✓	Off	12	1	Unlimited	1	100%		✓	0




```
================================================================================
=== Instance: /ALU_top/alu_inst
=== Design Unit: work.ALU_if
================================================================================
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         34        34         0   100.00%

================================Toggle Details================================
```

```
================================================================================
=== Instance: /ALU_top/dut
=== Design Unit: work.ALU_4_bit
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         6         6         0   100.00%

================================Branch Details================================
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       8         8         0   100.00%

================================Statement Details================================

Statement Coverage for instance /ALU_top/dut --
```

```
DIRECTIVE COVERAGE:
--------------------------------------------------------------------------------------------
Name                                      Design Design   Lang File(Line)      Hits Status
                                          Unit   UnitType
--------------------------------------------------------------------------------------------
/ALU_top/dut/ALU_if_inst/reset_coverage  ALU_sva Verilog  SVA  ALU_sva.sv(20)     2 Covered
/ALU_top/dut/ALU_if_inst/alu_add_coverage
                                          ALU_sva Verilog  SVA  ALU_sva.sv(27)    16 Covered
/ALU_top/dut/ALU_if_inst/alu_Sub_coverage
                                          ALU_sva Verilog  SVA  ALU_sva.sv(34)     6 Covered
/ALU_top/dut/ALU_if_inst/alu_Not_A_coverage
                                          ALU_sva Verilog  SVA  ALU_sva.sv(41)     6 Covered
/ALU_top/dut/ALU_if_inst/alu_ReductionOR_B_coverage
                                          ALU_sva Verilog  SVA  ALU_sva.sv(48)     4 Covered

TOTAL DIRECTIVE COVERAGE: 100.00%  COVERS: 5

ASSERTION RESULTS:
--------------------------------------------------------------------
Name                 File(Line)                   Failure      Pass
                                                  Count        Count
--------------------------------------------------------------------
/ALU_top/dut/ALU_if_inst/reset_assertion
                     ALU_sva.sv(19)                     0          1
/ALU_top/dut/ALU_if_inst/alu_add_assertion
                     ALU_sva.sv(26)                     0          1
/ALU_top/dut/ALU_if_inst/alu_Sub_assertion
                     ALU_sva.sv(33)                     0          1
/ALU_top/dut/ALU_if_inst/alu_Not_A_assertion
                     ALU_sva.sv(40)                     0          1
/ALU_top/dut/ALU_if_inst/alu_ReductionOR_B_assertion
                     ALU_sva.sv(47)                     0          1

Total Coverage By Instance (filtered view): 99.64%
```

3

```
module arbiter();

    logic request, grant, clk;
    logic frame, irdy;

    // Upon rising of the signal “request” by a master, the arbiter should raise the “grant” within 2 to 5
    // clock cycles.
    property SVA_1;
        @(posedge clk) $rose(request) |-> ##[2:5] grant;
    endproperty

    assert property(SVA_1);

    // Once the “grant” is raised, the master should acknowledge acceptance in the same clock cycle
    // by lowering the "frame" and "irdy" signals after they were high in the previous cycle.
    property SVA_2;
        @(posedge clk) $rose(grant) |-> $fell(frame) && $fell(irdy);
    endproperty

    assert property(SVA_2);

    // Once the master completes the transaction it raises the "frame" and "irdy" signals, followed by
    // that, the arbiter should lower the "grant" signal on the next clock cycle after it was high.
    property SVA_3;
        @(posedge clk) ($rose(frame) && $rose(irdy)) |=> $fell(grant);
    endproperty

    assert property(SVA_3);

endmodule
```

4

```
module FSM();

    logic [2:0] cs;
    logic clk;
    logic get_data;

    // Current state internal signal “cs” will always stay one-hot irrespective of the input conditions.
    property SVA_1;
        @(posedge clk) 1 |-> $onehot(cs);
    endproperty

    assert property(SVA_1);

    // If the current state “cs” is "IDLE" and if "get_data" input is raised, then the state is
    // "GEN_BLK_ADDR" the following cycle and 64 cycles later the state should change to "WAITO."
    property SVA_2;
        @(posedge clk) (cs == IDLE && $rose(get_data)) |=> (cs == GEN_BLK_ADDR) ##64 (cs == WAIT0);
    endproperty

    assert property(SVA_1);

endmodule
```